Laboratorio 5. Canal inseguro.

1. Qué vamos a comprobar

Vamos a probar un canal HTTP entre un cliente y un servidor que se ejecutan en el mismo computador. El cliente utiliza curl para enviar un mensaje en formato JSON. El servidor, implementado con Node.js, lo recibe en la dirección `127.0.0.1:18080/mensaje` y devuelve una confirmación. Todos los datos utilizados son ficticios.

El servidor se crea con el módulo `node:http`, incluido en Node.js. Escucha en el puerto 18080 y atiende solicitudes `POST /mensaje`. Al recibir el JSON, comprueba que tenga los campos `usuario` y `mensaje` y responde con un estado 200 y el contenido recibido.

Por qué es inseguro

Este canal utiliza HTTP sin TLS, por lo que el contenido viaja sin cifrar. Quien pueda capturar el tráfico de ese trayecto puede reconstruir la conversación y leer los mensajes. Esta exposición puede aprovecharse mediante captura de tráfico, también llamada sniffing. En un sistema que enviara información sensible por ese canal, podrían quedar expuestos datos privados.

En la práctica observaremos tráfico local con Wireshark. La dirección `127.0.0.1` corresponde a loopback: la comunicación vuelve al mismo computador y no sale por Wi-Fi. La demostración consiste en leer el mensaje desde la captura; no implica intervenir una red ajena ni modificar lo enviado.

Herramientas y archivos

Se necesitan Node.js 22 o posterior, curl y Wireshark. En Windows, Wireshark debe contar con Npcap para capturar el tráfico de loopback.

El ZIP está en el repositorio público `SWARCH-1A/streaming`, en la rama `feature/lab-05-secure-channel-g1`, dentro de labs/secure-channel/docs/Grupo-1-HTTP-Wireshark.zip. Contiene esta guía y los archivos del ejemplo. Tras extraerlo, utilizaremos la carpeta `ejemplo-http` para iniciar el servidor y enviar la petición.

Archivo	Para qué sirve
`server.mjs`	Inicia el servidor HTTP en `127.0.0.1:18080`.
`handler.mjs`	Recibe el JSON, comprueba sus campos y prepara la respuesta.
`solicitud.json`	Contiene el usuario y el mensaje ficticios que enviará curl.

El mensaje incluye el texto `LAB_CANAL_INSEGURO`, que nos permitirá reconocerlo dentro de la captura.

2. Iniciar el servidor

Abre la carpeta extraída donde se encuentran `ejemplo-http`, `docs` y `README.md`. Haz clic en la barra de dirección del Explorador, escribe `powershell` y pulsa Enter. La terminal se abrirá en esa carpeta. Comprueba las herramientas:

```
node --version
curl.exe --version
```

Después inicia el servidor:

```
node ejemplo-http/server.mjs
```

Debe aparecer:

```
Servidor del laboratorio: http://127.0.0.1:18080/mensaje
Déjalo abierto. Para terminar, pulsa Ctrl+C en esta terminal.
```

Deja esta terminal abierta. El programa permanece a la espera de peticiones, por lo que no volverá a mostrar el indicador habitual para escribir comandos. Abre una segunda ventana de PowerShell desde la misma carpeta; la utilizaremos después para ejecutar curl.

3. Preparar la captura en Wireshark

1. Abre Wireshark. Si hay una captura en curso, detenla con el cuadrado rojo de la barra superior.
2. Entra en **Captura** → **Opciones** o pulsa **Ctrl+K**.
3. En la pestaña **Entrada**, selecciona la fila **Adapter for loopback traffic capture** con **un solo clic**. No mantengas pulsadas Ctrl ni Shift. Comprueba que solo esa fila quede seleccionada; las casillas de la columna «Promiscuo» no sirven para seleccionar la interfaz.
4. En el campo inferior **Filtro de captura para interfaces seleccionadas**, escribe:

```
tcp port 18080
```

5. No pulses Enter. Pulsa **Tab** para salir del campo y comprobar que el filtro no aparece como inválido.
6. Guarda una imagen de esta ventana en la que se vean la interfaz seleccionada y el filtro.
7. Pulsa **Iniciar**, en la parte inferior derecha. Deja que Wireshark capture mientras envías la petición.

El doble clic sobre una interfaz puede iniciar la captura directamente. Por eso seleccionamos con un solo clic y usamos el botón Iniciar cuando la configuración está lista. El filtro limita la captura al puerto del ejemplo.

4. Enviar la petición y guardar los paquetes

En la segunda terminal, situada en la misma carpeta, ejecuta:

```
curl.exe --noproxy "*" --http1.1 --include `
  --header "Content-Type: application/json" `
  --data-binary "@ejemplo-http/solicitud.json" `
  http://127.0.0.1:18080/mensaje
```

Copia el bloque completo. El acento grave al final de las tres primeras líneas permite continuar el comando en PowerShell; no debe tener espacios después. Curl lee `solicitud.json` y lo envía mediante POST. La opción `--include` muestra las cabeceras de la respuesta y `--noproxy` evita que la petición local pase por un proxy configurado en el equipo.

La respuesta debe comenzar por `HTTP/1.1 200 OK` e incluir estos datos:

```
{
  "recibido": true,
  "usuario": "estudiante-demo",
  "mensaje": "LAB_CANAL_INSEGURO: mensaje de prueba para el laboratorio."
}
```

El estado 200 indica que el servidor atendió la petición. No demuestra que el canal esté protegido.

Vuelve a Wireshark y sigue estos pasos:

1. Comprueba que aparecieron paquetes y pulsa el **cuadrado rojo de Detener** en la barra superior.
2. Abre **Archivo** → **Guardar como** y guarda la captura con el nombre `canal-inseguro.pcapng`.
3. En la barra de filtro situada encima de la lista de paquetes, escribe y pulsa Enter:

```
tcp.port == 18080 && http
```

4. Busca las filas `POST /mensaje HTTP/1.1` y `HTTP/1.1 200 OK`. Guarda una segunda imagen con el filtro y ambas filas visibles.

Este es un filtro de visualización: selecciona cuáles de los paquetes guardados se muestran. Su sintaxis es distinta de la del filtro de captura utilizado antes de iniciar.

Si la lista queda vacía, prueba `tcp.port == 18080`. Si aparecen paquetes TCP, selecciona uno y entra en **Analizar** → **Decodificar como / Analyze** → **Decode As**. Elige HTTP para el puerto 18080, acepta y vuelve a aplicar el filtro completo.

5. Leer el contenido del canal

1. Selecciona la fila `POST /mensaje` y pulsa el botón derecho.
2. Abre **Seguir → Flujo TCP / Follow → TCP Stream**.
3. Selecciona **Conversación completa / Entire conversation** y **Mostrar datos como: ASCII / Show data as: ASCII**.
4. Busca la línea `POST /mensaje HTTP/1.1`, el usuario `estudiante-demo` y el texto `LAB_CANAL_INSEGURO` dentro del JSON.
5. Localiza también `HTTP/1.1 200 OK` y el JSON de respuesta, que debe contener `"recibido":true` y el mismo mensaje.
6. Guarda una tercera imagen con la solicitud y la respuesta legibles. Amplía la ventana o toma dos imágenes consecutivas si no cabe toda la conversación.

Wireshark reconstruye la conversación a partir de los paquetes capturados. Si puedes leer el mensaje desde esa ventana, has comprobado que su contenido quedó expuesto en el trayecto observado. JSON organiza los datos y TCP los transporta; ninguno de los dos aporta cifrado por sí mismo.

El observador necesita acceso al trayecto del tráfico. Capturar loopback en el propio computador no demuestra que cualquier persona conectada al mismo Wi-Fi pueda leer todas las conversaciones de los demás.

Para terminar, vuelve a la terminal del servidor y pulsa **Ctrl+C**. Conserva el archivo `.pcapng`: permite abrir y revisar la conversación después, aunque el servidor esté apagado.

6. Problemas frecuentes

Lo que ocurre	Qué revisar
`node` no se reconoce	Comprueba que Node.js esté instalado y abre una terminal nueva.
Aparece `EADDRINUSE`	El puerto 18080 ya está ocupado. Si es otra ejecución de esta práctica, reutilízala o detenla con Ctrl+C.
Curl no encuentra el archivo JSON	La terminal debe estar en la carpeta que contiene `ejemplo-http`.
Curl no conecta	Revisa que el servidor siga funcionando y que la dirección termine en `:18080/mensaje`.
No aparece la interfaz de loopback	Revisa la instalación de Npcap en Windows.
No aparecen paquetes	Inicia una captura en loopback y después vuelve a enviar la petición.

Para repetir el ensayo, inicia el servidor, prepara una captura nueva y envía otra vez el mismo archivo JSON.